

Advanced Data Stores

Phase 3 · Advanced · Estimated time: 2 hours

1. Concept From First Principles

Phase 1 introduced NoSQL categories at a survey level; Phase 2 Chapter 14 argued for polyglot persistence in the abstract. This chapter goes deep on the specific advanced data stores that show up constantly in senior-level system design interviews: wide-column stores like Cassandra, and the storage-engine idea — the LSM tree — that explains why they behave so differently from the relational databases you've used your whole career.

Wide-column stores: Cassandra

A **wide-column store** like Apache Cassandra organizes data around a **partition key** (which determines which node in the cluster owns the data, using consistent hashing, Phase 2 Chapter 7 directly) and an optional **clustering key** (which determines the sort order of rows within that partition). Unlike a relational table, rows in the same table don't need identical columns — each row can have a different set of columns, which is where the "wide-column" name comes from. Cassandra is explicitly designed leaderless (Phase 2 Chapter 5) and quorum-based, deliberately choosing availability over strict consistency (an AP system in Chapter 4's CAP terms), and is built to absorb enormous, sustained write throughput — production Cassandra clusters handling millions of writes per second are common.

Backend engineer analogy

If you've designed a MySQL table where you had to add nullable columns for optional fields, and dealt with the awkwardness of many NULL values wasting space and complicating queries, Cassandra's per-row flexible columns solve that specific pain directly — each row only stores the columns it actually has, at the cost of losing the rigid, enforced schema and multi-row transaction guarantees relational databases give you.

The LSM tree: why wide-column stores write so fast

The mechanical reason Cassandra-style databases handle such high write volume is their storage engine: the **Log-Structured Merge tree (LSM tree)**, a fundamentally different approach from the B-tree structure most relational databases use. Writes are first appended to an in-memory structure (a **memtable**) and a durable on-disk write-ahead log, both very fast, sequential operations — no in-place updates to existing on-disk data structures, which is what makes B-tree writes comparatively slower under heavy write load, since a B-tree write often means locating and modifying a specific on-disk page. Once the memtable fills up, it's flushed to disk as an immutable file called an **SSTable (Sorted String Table)**. Over time, a background process called **compaction** merges multiple SSTables together, removing overwritten and deleted data (marked with special **tombstone** records rather than deleted in place) to keep read performance and storage usage under control.

This design creates a real, honest trade-off, and interviewers value hearing it stated plainly: LSM-tree writes are extremely fast (sequential, append-only), but reads can be slower than a B-tree's, because a

single read might need to check the memtable and potentially several SSTables before finding the most recent version of a value (mitigated in practice by **bloom filters**, Chapter 10, which let Cassandra quickly skip SSTables that definitely don't contain a given key). This is precisely why the honest guidance for Cassandra is: use it when you know your query patterns up front and need very high write throughput, and avoid it when you need flexible, ad-hoc queries, multi-row transactions, or joins — capabilities it deliberately doesn't provide.

Time-series databases and Elasticsearch, briefly

Time-series databases (InfluxDB, TimescaleDB) are purpose-built for data that's overwhelmingly append-only and queried by time range — metrics, sensor readings, application logs — using storage layouts and compression techniques optimized specifically for this access pattern (often achieving dramatically better compression than a general-purpose database, since consecutive time-series values tend to be similar to each other). This connects directly forward to Chapter 13's observability chapter, since metrics storage is exactly this workload. Elasticsearch, covered in depth in Phase 2 Chapter 13 for full-text search, is worth re-mentioning here as part of the broader "advanced data stores" toolkit: it's built on Lucene, uses an inverted index rather than a B-tree or LSM tree, and, like Cassandra, is horizontally sharded and generally not your system of record.

The unifying lesson across this chapter, tying back to Phase 2 Chapter 14's polyglot persistence argument: every one of these advanced stores made a deliberate architectural trade-off — Cassandra traded joins/transactions for write throughput and availability; time-series databases traded generality for compression and time-range query speed; Elasticsearch traded strong consistency for full-text search speed. None of them is a general-purpose replacement for a relational database; each is the right tool for a specific, well-understood access pattern, and naming that pattern explicitly is what a strong interview answer sounds like, versus a memorized "use Cassandra for scale" cliché.

2. Real-World Example / Case Study

A large-scale messaging or activity-feed feature at Swiggy- or Flipkart-scale — storing billions of "user X did Y at time Z" events with extremely high write volume and simple, known query patterns ("get recent activity for user X") — is a strong Cassandra fit: writes dominate, the query pattern is fixed and partition-key-friendly, and occasional slightly-stale reads are entirely acceptable.

Careem's and Talabat's real-time driver-location and trip-telemetry pipelines, generating continuous high-frequency GPS pings, are a natural time-series database fit — the query pattern ("show this driver's location history over the last hour") is inherently time-range-based, and the compression advantages of a purpose-built time-series store meaningfully reduce storage costs at that data volume.

Razorpay's or PhonePe's fraud-detection and monitoring systems, needing to store and query enormous volumes of transaction metrics and system telemetry, typically combine a time-series database for metrics with Elasticsearch for log search — a concrete, real illustration of polyglot persistence applying several advanced stores side by side, each doing the one job it's actually built for.

3. Diagram

LSM Tree write path (Cassandra-style):

```
WRITE --> [ Write-Ahead Log (disk, durable) ]
      --> [ Memtable (in-memory, sorted) ]
          |
      (memtable fills up)
          v
      [ SSTable-1 (disk, immutable) ]
      [ SSTable-2 (disk, immutable) ]
      [ SSTable-3 (disk, immutable) ]
          |
      background COMPACTION merges + removes tombstones
          v
      [ Fewer, larger, cleaner SSTables ]
```

READ for key "user:42":
check Memtable -> check SSTable-3 (newest) -> SSTable-2 -> SSTable-1
(Bloom filters, Ch 10, let Cassandra SKIP SSTables that can't have the key)

4. Trade-offs Table

Store	Optimized for	Weak for
Cassandra (wide-column, LSM)	Very high write throughput, known query patterns	Ad-hoc queries, joins, multi-row transactions
Relational (B-tree)	Flexible queries, joins, ACID transactions	Extreme write throughput at massive scale
Time-series DB (InfluxDB/TimescaleDB)	Time-range queries, high compression	General-purpose querying outside time-range patterns
Elasticsearch (Lucene, inverted index)	Full-text search, relevance ranking	System-of-record durability, ACID guarantees

5. Common Interview Questions

Q1: "Why does Cassandra write so much faster than a typical relational database at high volume?"

Model approach: Explain the LSM tree's append-only, sequential-write design (write-ahead log + memtable) versus a B-tree's in-place page updates, and name the read-side cost (checking multiple SSTables) as the honest trade-off being made for that write speed.

Q2: "When would you NOT use Cassandra, even at high scale?"

Model approach: Name the concrete missing capabilities — joins, ad-hoc queries not matching your partition key, multi-row ACID transactions — and explain that Cassandra requires knowing your query patterns upfront, unlike a relational database's flexibility.

Q3: "What is compaction, and why is it necessary?"

Model approach: Describe compaction as the background process merging multiple immutable SSTables into fewer, cleaner ones, removing overwritten data and tombstones, and explain that without it, reads would progressively slow down as the number of SSTables to check grows unbounded.

Q4: "Design the storage layer for a fleet-tracking system logging GPS pings every 2 seconds from 50,000 vehicles."

Model approach: Recommend a time-series database or a Cassandra table keyed by vehicle_id + timestamp, justify with the write-volume math (50,000 vehicles x 0.5 writes/sec = 25,000 writes/sec sustained), and note the time-range query pattern this workload naturally has.

6. Practice Exercises

- 1 Design a Cassandra schema (partition key + clustering key) for storing a user's order history, optimized for the query 'get this user's most recent 20 orders.'
- 2 Explain, in your own words, why a bloom filter (Chapter 10) speeds up Cassandra reads specifically, connecting it to the SSTable-checking process in this chapter's diagram.
- 3 Propose a polyglot persistence architecture for a food delivery app's backend that uses at least one relational database, Cassandra (or similar), a time-series database, and Elasticsearch, naming what each stores.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Why does NoSQL exist? (MongoDB, Cassandra) System Design"	Good framing of why wide-column stores exist and what problem they solve.
English	"Intro to NoSQL with Apache Cassandra"	Hands-on introduction to Cassandra's data model and architecture.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers advanced data stores as part of a full bilingual course.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Places Cassandra/LSM trees in context of the broader Hindi HLD roadmap.

8. Key Takeaways

- Wide-column stores like Cassandra organize data by partition key (consistent hashing) and clustering key.
- Cassandra is leaderless, quorum-based, and deliberately AP (Chapter 4), trading strict consistency for availability and write throughput.
- The LSM tree writes sequentially (memtable + WAL, then SSTables), making writes far faster than a B-tree at scale, at some read-side cost.
- Compaction merges SSTables and removes tombstones to keep reads and storage usage under control over time.
- Time-series databases optimize specifically for time-range queries and high compression on append-only data.
- Every advanced store here made a deliberate trade-off; none is a general-purpose relational database replacement.

- Polyglot persistence (Phase 2 Ch 14) is how real systems combine several of these stores, each for its own strength.